

Spring Boot 4 테스트 환경 마이그레이션 가이드

이 문서는 JHipster 9에서 제공하는 Spring Boot 4 기반 테스트 환경을 기존 프로젝트에 적용하는 방법을 안내합니다. 기존 프로젝트는 Spring Boot 4.0.4와 Testcontainers를 사용하지만, 레거시한 커스텀 컨텍스트 팩토리 (`SqlTestContainersSpringContextCustomizerFactory` 등)에 의존하고 있어 클래스명이 변경되면 `ClassNotFoundException` 이 발생할 수 있습니다. JHipster 9에서는 이러한 패턴이 간소화되었으며, Spring Boot 4.1+에서 제공하는 `spring-boot-testcontainers` 모듈과 `@ServiceConnection` 을 사용하면 커스텀 팩토리 없이도 Testcontainers를 사용할 수 있습니다. 이 가이드는 테스트 코드를 정비하여 다른 개발자(에이전트)도 쉽게 작업할 수 있도록 구성되어 있습니다.

1. Maven 의존성 업데이트

1. `spring-boot-testcontainers` 모듈 추가 – Spring Boot 4에서 Testcontainers와의 통합을 지원하는 모듈입니다. `pom.xml` 의 `<dependencies>` 섹션에 다음을 추가합니다.

```
<!-- Testcontainers 자동 연동을 위해 추가 -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-testcontainers</artifactId>
  <scope>test</scope>
</dependency>
```

1. 기존 Testcontainers 의존성 정리 – `testcontainers`, `testcontainers-junit-jupiter`, `testcontainers-oracle-xe` 는 유지하지만, `testcontainers-jdbc` 는 사용하지 않으므로 제거할 수 있습니다. `pom.xml` 에선 현재 테스트 관련 의존성이 다음과 같이 정의되어 있습니다 ¹. 필요 없는 모듈을 삭제하고, 새로운 모듈을 추가하세요.

```
<!-- spring-boot-starter-test는 유지 -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-test</artifactId>
  <scope>test</scope>
</dependency>

<!-- Web MVC 테스트 지원 유지 -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-webmvc-test</artifactId>
  <scope>test</scope>
</dependency>

<!-- 필요 없는 JDBC 모듈 제거 -->
<!-- <dependency>
  <groupId>org.testcontainers</groupId>
  <artifactId>testcontainers-jdbc</artifactId>
```

```

    <scope>test</scope>
</dependency> -->

<!-- 필수 Testcontainers 모듈 유지 -->
<dependency>
    <groupId>org.testcontainers</groupId>
    <artifactId>testcontainers</artifactId>
    <scope>test</scope>
</dependency>
<dependency>
    <groupId>org.testcontainers</groupId>
    <artifactId>testcontainers-junit-jupiter</artifactId>
    <scope>test</scope>
</dependency>
<dependency>
    <groupId>org.testcontainers</groupId>
    <artifactId>testcontainers-oracle-xe</artifactId>
    <scope>test</scope>
</dependency>

```

1. **Mockito 에이전트 설정 유지** - JDK 21에서 Mockito 경고를 없애기 위해 `maven-surefire-plugin` 에서 `-javaagent` 설정을 유지합니다
2. 새 기능을 도입하더라도 이 설정은 변경하지 않습니다.

2. 레거시 테스트 인프라 제거

Spring Boot 4 + `spring-boot-testcontainers` 를 사용할 경우 다음 레거시 파일과 메타데이터는 삭제합니다. 다른 에이전트가 중복 구현하지 않도록 반드시 제거하세요.

- `src/test/resources/META-INF/spring.factories` - `SqlTestContainersSpringContextCustomizerFactory` 와 `RedisTestContainersSpringContextCustomizerFactory` 등록을 삭제합니다.
- `src/test/java/com/daangcool/stack/config/SqlTestContainersSpringContextCustomizerFactory.java` - 리플렉션을 이용해 `TestContainer` 를 로딩하는 클래스이며, Spring Boot 4에서는 필요 없습니다.
- `src/test/java/com/daangcool/stack/config/RedisTestContainersSpringContextCustomizerFactory.java` - Redis 컨테이너를 동적으로 주입하는 커스텀 팩토리로, `@ServiceConnection` 또는 `@ImportTestcontainers` 로 대체합니다.
- `src/test/java/com/daangcool/stack/config/TestContainer.java`, `src/test/java/com/daangcool/stack/config/SqlTestContainer.java`, `src/test/java/com/daangcool/stack/config/RedisTestContainer.java`, `src/test/java/com/daangcool/stack/config/EmbeddedSQL.java`, `src/test/java/com/daangcool/stack/config/EmbeddedRedis.java` - 기존 커스텀 애너테이션과 컨테이너 인터페이스를 더 이상 사용하지 않습니다.

3. 통합 테스트 애너테이션 재작성

기존 `IntegrationTest` 애너테이션은 `@EmbeddedRedis` 와 `@EmbeddedSQL` 을 사용해 커스텀 팩토리를 동작시켰습니다

3. 새 환경에서는 Spring Boot가 Testcontainers를 자동으로 관리하도록 다음과 같이 수정합니다.

```

package com.daangcool.stack;

import com.daangcool.stack.StackApp;
import com.daangcool.stack.config.LiquibaseTestConfiguration;
import com.daangcool.stack.config.AsyncSyncConfiguration;
import com.daangcool.stack.config.JacksonConfiguration;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.boot.testcontainers.service.connection.ServiceConnection;
import org.testcontainers.containers.OracleContainer;
import org.testcontainers.containers.GenericContainer;
import org.testcontainers.junit.jupiter.Container;
import org.testcontainers.junit.jupiter.Testcontainers;
import org.testcontainers.utility.DockerImageName;

/**
 * Base composite annotation for integration tests.
 */
@Target(ElementType.TYPE)
@Retention(RetentionPolicy.RUNTIME)
@SpringBootTest(classes = {StackApp.class, LiquibaseTestConfiguration.class,
    JacksonConfiguration.class, AsyncSyncConfiguration.class})
@Testcontainers
public @interface IntegrationTest {

    // Oracle 데이터베이스 컨테이너 - Spring Boot가 datasource 속성을 자동 설정
    @Container
    @ServiceConnection
    static OracleContainer oracle = new OracleContainer(
        DockerImageName.parse("gvenzl/oracle-xe:21-slim-faststart"))
        .withDatabaseName("ORCLTEST")
        .withUsername("testuser")
        .withPassword("testpass")
        .withReuse(true);

    // Redis 컨테이너 - Spring Boot가 Redis 속성을 자동 설정
    @Container
    @ServiceConnection
    static GenericContainer<?> redis = new GenericContainer<>(DockerImageName.parse("redis:
8.6.1"))
        .withExposedPorts(6379)
        .withReuse(true);
}

```

위 코드에서는 `@ServiceConnection` 을 통해 Testcontainers에서 생성된 데이터베이스/캐시의 접속 정보를 Spring 환경에 자동 주입합니다. 별도의 커스텀 팩토리나 `DynamicPropertySource` 메서드를 작성할 필요가 없습니다.

4. Redis 테스트 컨테이너 사용 시 대안

JHipster 9은 `ImportTestcontainers` 와 `DynamicPropertySource` 를 활용해 Redis 컨테이너를 정의하는 `RedisTestContainer` 인터페이스를 제공합니다. `@ServiceConnection` 을 사용할 수 없는 상황(예: Spring Boot 4.0.x 이하)에서는 다음과 같이 `ImportTestcontainers` 방식을 사용합니다.

```
// RedisTestContainer.java
package com.daangcool.stack.config;

import org.junit.jupiter.api.extension.ExtensionContext;
import org.springframework.test.context.DynamicPropertyRegistry;
import org.springframework.test.context.DynamicPropertySource;
import org.testcontainers.containers.GenericContainer;
import org.testcontainers.containers.output.Slf4jLogConsumer;
import org.testcontainers.junit.jupiter.Container;
import org.testcontainers.junit.jupiter.Testcontainers;
import org.slf4j.LoggerFactory;

@Testcontainers
public interface RedisTestContainer {

    @Container
    GenericContainer<?> redisContainer = new GenericContainer<>("redis:8.6.1")
        .withExposedPorts(6379)
        .withLogConsumer(new Slf4jLogConsumer(LoggerFactory.getLogger(RedisTestContainer.class)))
        .withReuse(true);

    @DynamicPropertySource
    static void registerProperties(DynamicPropertyRegistry registry) {
        registry.add("jhipster.cache.redis.server", () ->
            "redis://" + redisContainer.getHost() + ":" + redisContainer.getMappedPort(6379));
    }
}

// IntegrationTest.java
@SpringBootTest(classes = {StackApp.class, LiquibaseTestConfiguration.class,
    JacksonConfiguration.class, AsyncSyncConfiguration.class})
@ImportTestcontainers(RedisTestContainer.class)
public @interface IntegrationTest {}
```

이 방식은 Spring Boot 4.0.x에서도 사용할 수 있으며, `redisContainer.getHost() / getMappedPort()` 를 사용해 최신 Testcontainers API를 따릅니다. 그러나 가능하다면 `@ServiceConnection` 방식이 더 간단합니다.

5. 테스트 프로파일 구성

기존 `EmbeddedSQL` 애너테이션은 테스트 프로파일을 `testdev`, `test` 로 설정했습니다⁴. 새 환경에서는 `@ActiveProfiles` 를 테스트 클래스에 직접 선언하거나, `IntegrationTest` 에 다음과 같이 추가해 공통 프로파일을 설정할 수 있습니다.

```
@Target(ElementType.TYPE)
@Retention(RetentionPolicy.RUNTIME)
@SpringBootTest(/* ... */)
@ActiveProfiles({"test"})
@Testcontainers
public @interface IntegrationTest { /* ... */ }
```

프로젝트에서 사용하는 `application-test.yml` 이나 `application-testdev.yml` 이 있다면 이 프로파일 설정을 반영하세요.

6. 테스트 클래스 오더링(Optional)

JHipster 9의 `SpringBootTestClassOrderer` 는 통합 테스트에 `IntegrationTest` 애너테이션이 붙어 있는 경우 다른 테스트보다 나중에 실행되도록 정렬합니다. 테스트 실행 순서가 중요하다면 다음과 같이 `junit-platform.properties` 에 등록해 사용할 수 있습니다.

```
junit.jupiter.testclass.order.default=com.daangcool.stack.config.SpringBootTestClassOrderer
```

필수 사항은 아니며, 테스트 간 의존성이 없도록 구현하는 것을 우선합니다.

7. 마이그레이션 체크리스트

1. `spring-boot-testcontainers` 의존성을 추가하고 `testcontainers-jdbc` 의존성을 제거합니다.
2. `spring.factories` 와 커스텀 컨텍스트 팩토리(Java 파일)를 삭제합니다.
3. `IntegrationTest` 애너테이션을 새 구조로 재작성하고, `@ServiceConnection` 을 이용해 데이터베이스와 Redis 컨테이너를 선언합니다.
4. 테스트 프로파일을 `@ActiveProfiles` 로 명시하거나, `application-test.yml` 설정을 확인합니다.
5. 필요하다면 JUnit 클래스 오더러(`SpringBootTestClassOrderer`)를 등록합니다.

이 가이드를 따라 기존 프로젝트의 테스트 환경을 Spring Boot 4 표준에 맞게 현대화하면 클래스 이름 변경에 따른 `ClassNotFoundException` 문제를 예방하고, 다른 개발자(에이전트)들도 일관된 구조에서 테스트를 작성할 수 있습니다.

1 2 pom.xml

<https://github.com/Dangkoolwat/stack-app-2025-v1/blob/main/pom.xml>

3 IntegrationTest.java

<https://github.com/Dangkoolwat/stack-app-2025-v1/blob/main/src/test/java/com/daangcool/stack/IntegrationTest.java>

4 EmbeddedSQL.java

<https://github.com/Dangkoolwat/stack-app-2025-v1/blob/main/src/test/java/com/daangcool/stack/config/EmbeddedSQL.java>